



Department of Computer Science and Engineering
University of Dhaka

CSE 2216: Application Development Lab
Batch-29 (2nd Year 2nd Semester 2025)

Assignment 2: Project Progress

Team Name : Team Dev_Mavericks

Team Members

Sumaiya Rahman Soma(07)
Mir. Md. Ishrak Faisal (12)
Jobaer Hossain Tamim(24)
Chowdhury Shafahid Rahman (55)

Submitted on
July 5,2025

Idea :

'Murir Tin' is a smart local bus app that uniquely combines color-coded routes, live GPS tracking, QR-based easy ticket booking, a complaint box to submit, track and view complaints and emergency features — making daily transportation safer, smoother, and easier .

Tools Used:

- **Framework:** Flutter
- **Backend:** FastAPI + Supabase
- **Database & Authentication:** Supabase
- **Map & Location Services:** Mapbox API

App Screens:

1) Splash Screen, Sign Up and Login Screen:

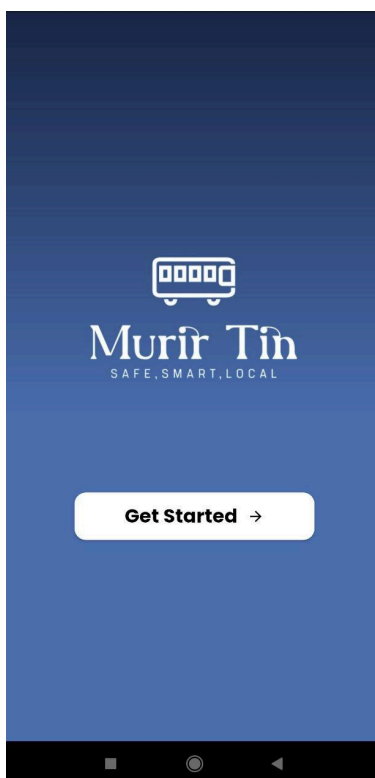


Fig: Splash Screen

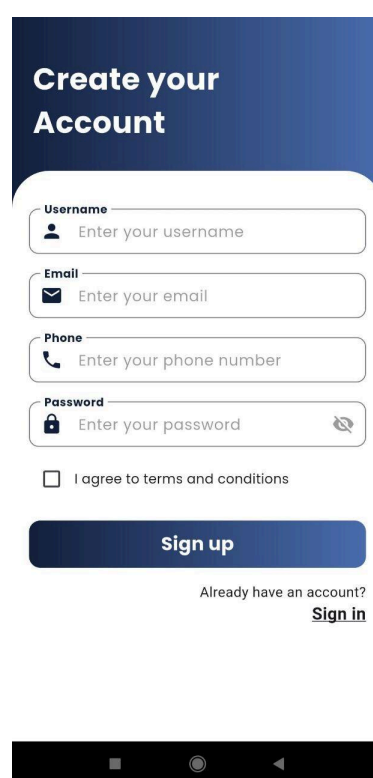


Fig: SignUp Screen

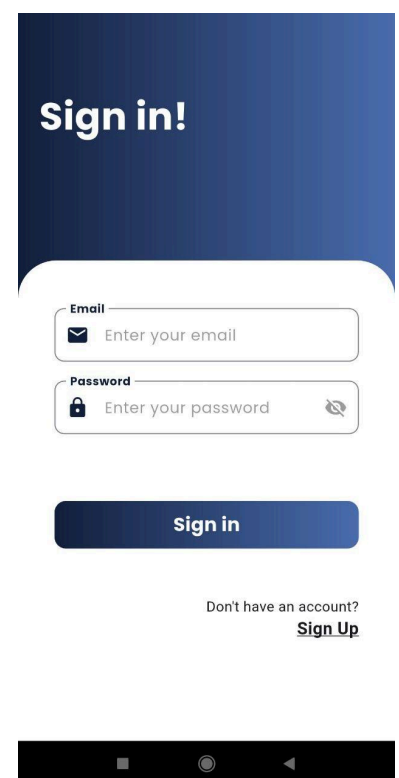


Fig: Sign In Screen

2) Landing page

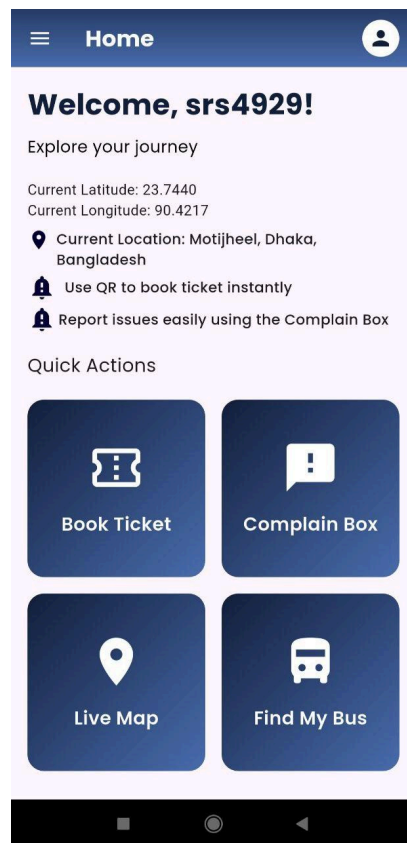


Fig:Landing page

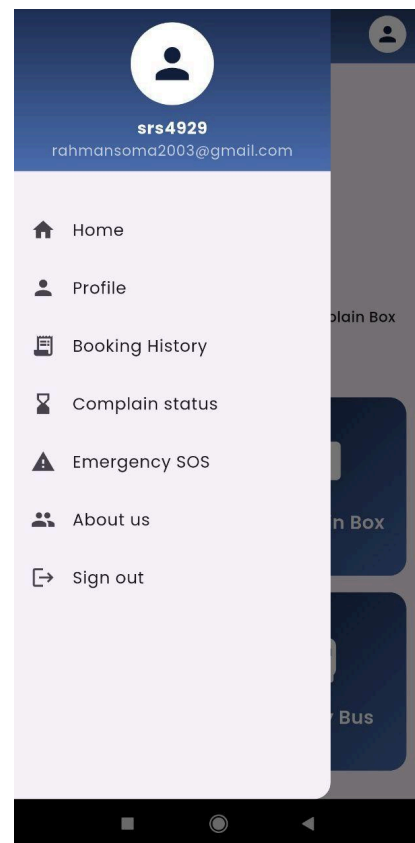


Fig:Sidebar

3) Book Ticket

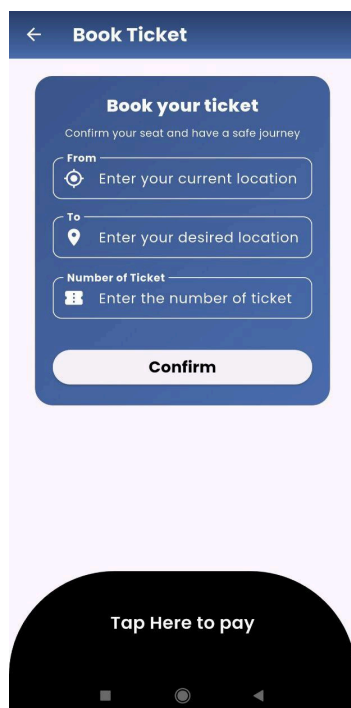


Fig: Select Route Screen

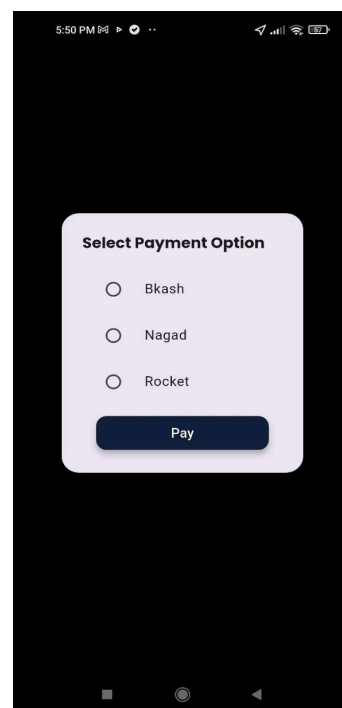


Fig:Payment Method Screen

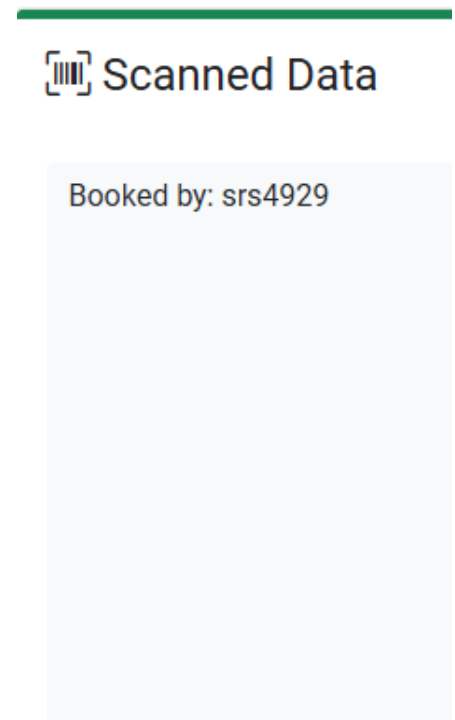
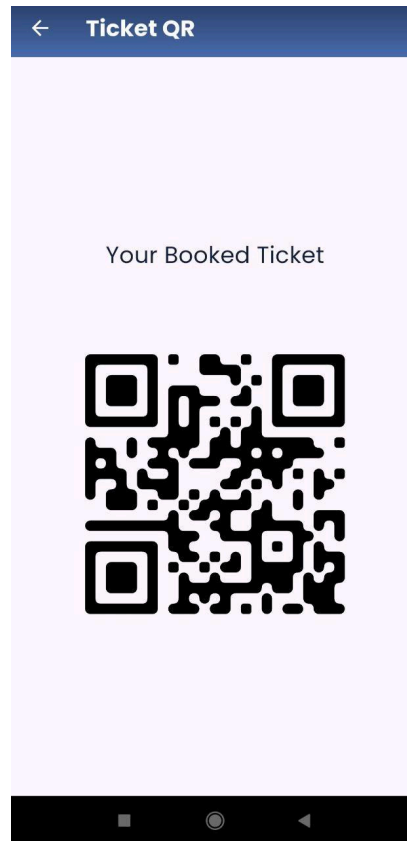
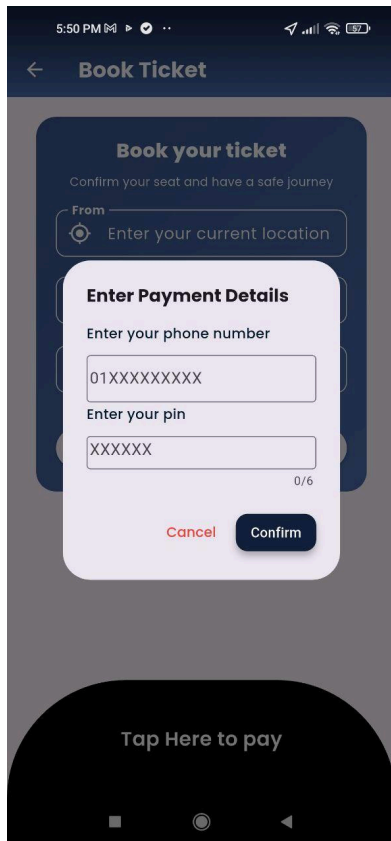
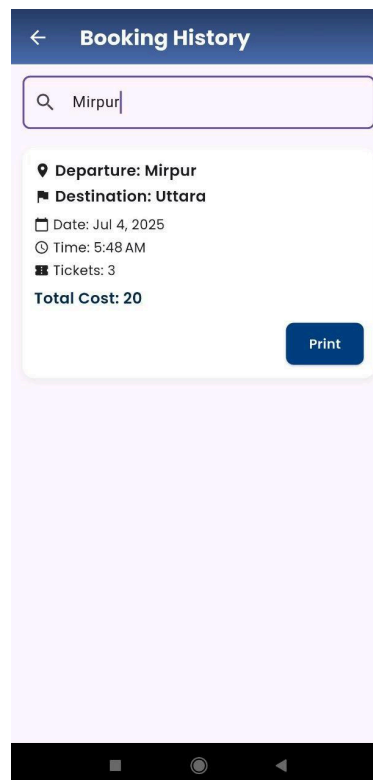
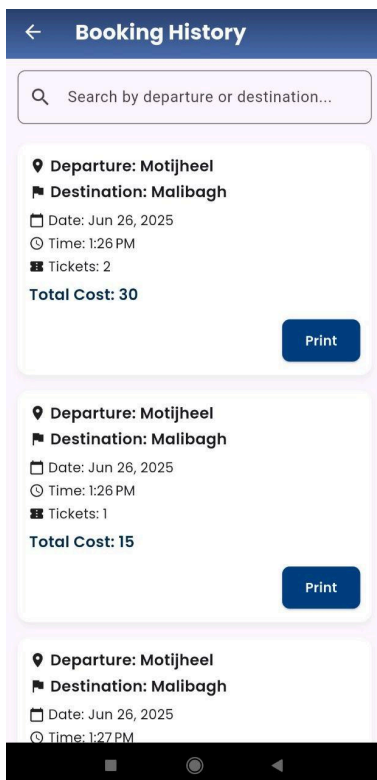


Fig: Phone & PIN Verification with QR Ticket Screen

4) Ticket Book history



5) Complain Box

The 'Complaint Box' app interface consists of two main screens. The left screen is the complaint form, featuring a dark blue header with a menu icon, the title 'Complaint Box', and a user profile icon. Below the header, there are three input fields: 'Company' with a dropdown menu showing 'Select Bus Company', 'Title' with a placeholder 'Give title of your complaint', and 'Description' with a placeholder 'Give details about your complaint'. A blue 'Submit' button is positioned at the bottom. The right screen shows the 'Company' dropdown menu expanded, displaying a list of bus companies: Akik, Alaka ATCL, Anabil Super (highlighted), Borak Transport, Dhaka Nagar Paribahan, Gazipur Paribahan, Hanif Metro Service, J.M. Turag / Great Turag, Khaja Baba Paribahan Ltd., Manjil, and Mega City. The bottom of both screens features a black navigation bar with three icons: a menu icon, a circle, and a back arrow.

Fig: Complaint Box with selecting Bus Company

6)Complaint Status

The 'Complaint Status' app interface consists of two main screens. The left screen, titled 'Complaint Status', displays a list of five complaint entries. Each entry includes the issue type, the date and time, and a button to 'View Current Status & Details'. The entries are: 'Driver Issue' (26/06/2025 03:45 AM), 'Fitness Issue' (25/06/2025 02:30 PM), 'Late Arrival' (24/06/2025 09:15 AM), 'Seat Issue' (23/06/2025 08:00 PM), and 'Bus Break Down' (22/06/2025 01:00 PM). The right screen, titled 'Current Status', shows a progress bar with three stages: 'Submitted' (checked), 'Accepted', and 'Solved'. Below the progress bar, there is a 'Complaint Details' section with a text box containing a long description of the complaint and a 'Back' button. The bottom of both screens features a black navigation bar with three icons: a menu icon, a circle, and a back arrow.

Fig:Complaint Status and Current Status

7) Social Issues

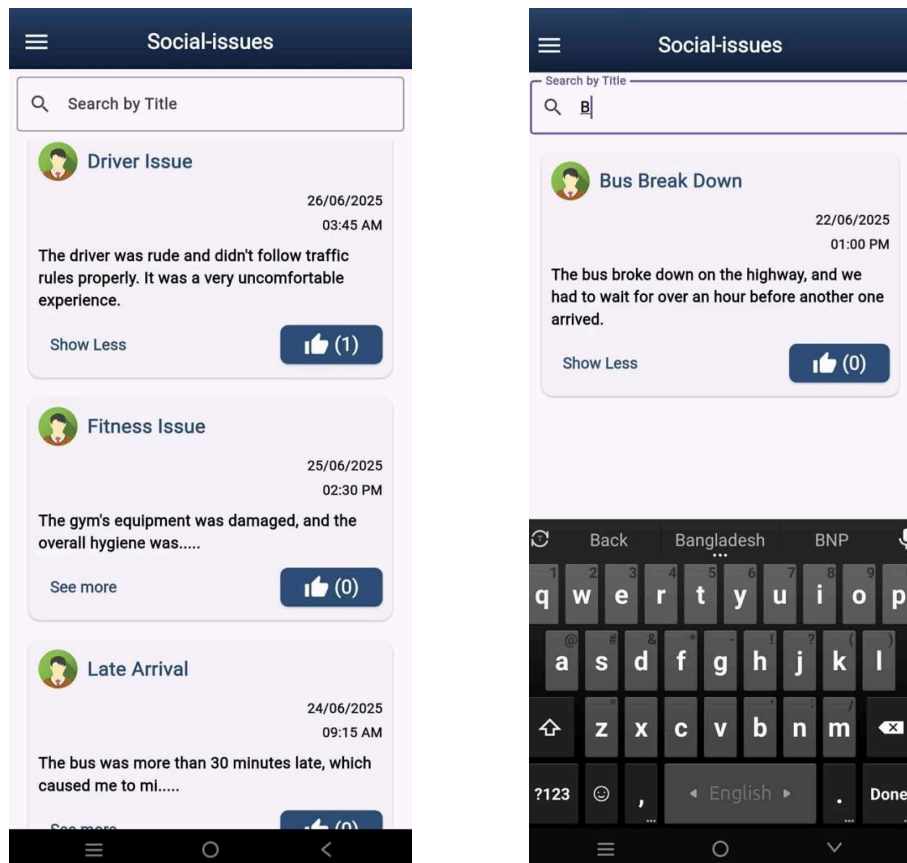
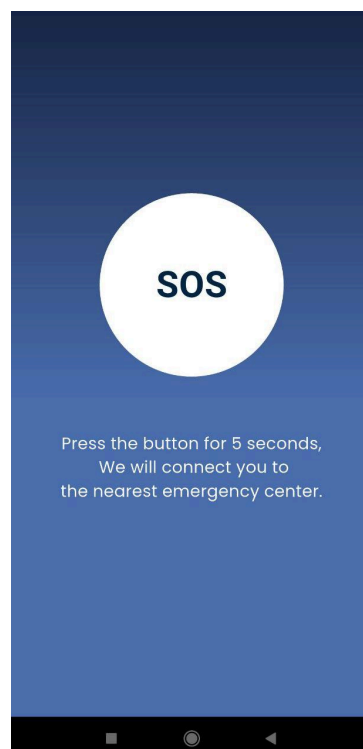


Fig: Social-issues with search bar

8) SOS Button



8) Find My Bus:

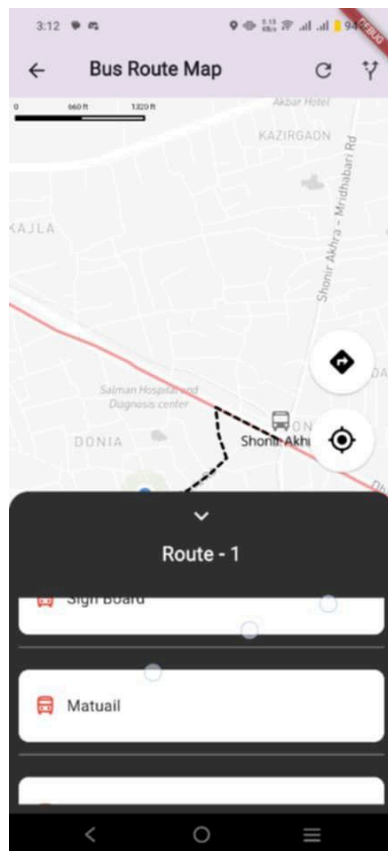


Fig: Route Layout

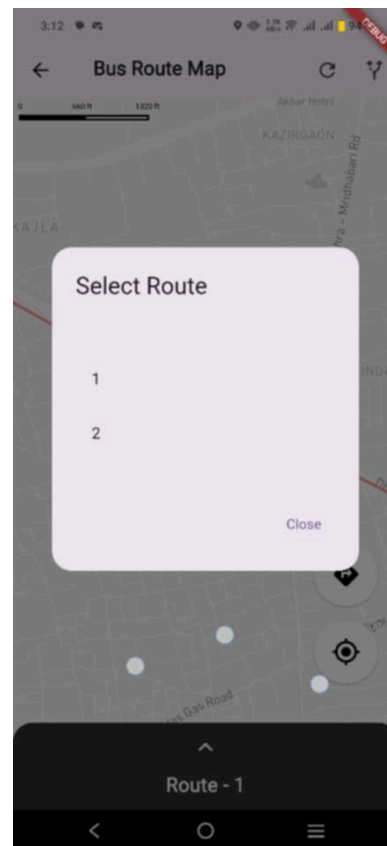


Fig:Route Selection

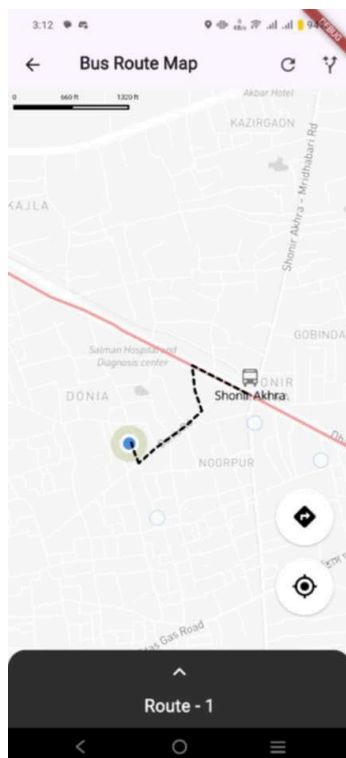


Fig:Direction Overview

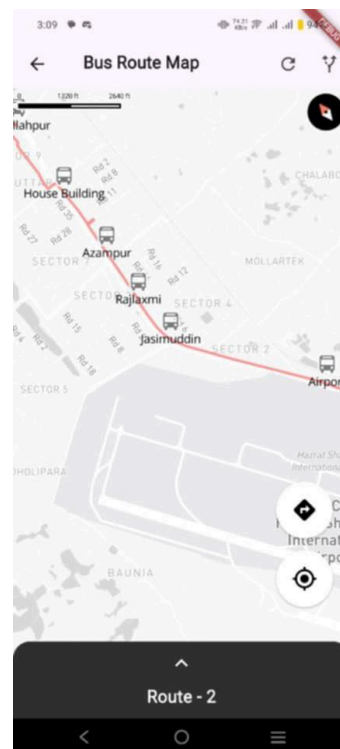


Fig: Stoppage Show

9) Profile:


My Profile



Ishrak

Full Name

Mir. Md. Ishrak Faisal

Email

ishrakfaisal100@gmail.com

Phone Number

01810773026

Edit Profile


Edit Profile



Change Picture

Username

Give your username

Full Name

Give your full name

Email

Give your email

Phone Number

Give your phone number

Update

Fig: My Profile and Edit Profile

10) Settings & About Us:


Settings


Current Password

Give your current password 

New Password

Give your new password 

Confirm New Password

Confirm your new password 

Change Password


About Us

Meet Our Team



Sumaiya Rahman Soma

CSEDU29

Roll: 07

rahmansoma2003@gmail.com



Md. Ishrak Faisal

CSEDU29

Roll: 12

ishrakfaisal100@gmail.com



Jobaer Hossain Tamim

CSEDU29

Roll: 24

jobaertamim7@gmail.com



Chowdhury Shafahid Rahman

CSEDU29

Roll: 55

shafahid66@gmail.com

Challenges and Solutions

Challenge 1: NDK Version Mismatch

Issue:

While setting up the Flutter project, we encountered an issue with the Android Native Development Kit (NDK). The installed NDK version did not match the version required by some dependencies, causing build errors and failed Gradle syncs during development.

Impact:

This mismatch blocked us from building and testing the app on Android emulators and physical devices, creating unexpected difficulties during development and testing phases. It also made it difficult to verify new features quickly on real devices.

Solution:

We resolved this by explicitly specifying the compatible NDK version in our `local.properties` file and aligning our Flutter and Gradle versions with official requirements. We also cross-checked the recommended NDK version in Android Studio's SDK Manager to maintain consistency across team members' environments.

Challenge 2: URL Launcher Configuration Issue

Issue:

We faced a problem when adding the `url_launcher` plugin to open external links. At first, we relied on LLM suggestions, but those didn't include the latest Android requirements.

Because of that, we missed adding the necessary `<queries>` block in our

AndroidManifest.xml file. For Android 11 (API level 30) and above, this part is now mandatory for the app to check for external apps safely.

Impact:

Without this our app sometimes failed to open external URLs — especially for actions like sending emails or sharing text. This broke parts of the user support flow during development. The links were not opened smoothly, which made the experience frustrating and slowed down testing too.

Solution:

We double-checked the official `url_launcher` docs and Android developer guidelines. Then, we updated our `AndroidManifest.xml` and added the `<queries>` section.

Challenge 3: Pinpointing Current Address Using Geolocator**Issue:**

We wanted to show the user's current address by getting their exact GPS location using the Geolocator plugin. At first, we tried to get the precise address for each user's location.

However, in some areas, especially with poor GPS signals or less mapped regions, the app couldn't find an exact address. This caused blank or incorrect address information to appear.

Impact:

This caused inconsistencies in the app's location-based features during development.

Solution:

To fix this, we decided to implement a fallback system: if the exact address wasn't found, the app shows the nearest available address instead.

This way, even if the pinpoint location isn't perfect, users still get a useful approximate address to proceed with. It improved overall user experience and reliability.

Challenge 4: PDF Download and Print Preview**Issue:**

We initially aimed to implement a Chrome-like PDF download experience within the app. For doing so, we followed advice from an LLM, but it didn't work out.

The method kept failing during development.

Impact:

Because of this, we couldn't get the PDF download feature ready on time. It slowed us down.

Solution:

Instead of relying on the LLM and facing multiple errors, we went back to YouTube tutorials and official documentation to understand how to do it properly. We then switched to saving PDFs in local storage, which worked better on different devices. We also added a print preview feature so users can see their tickets before printing.

Challenge 5: Mapbox integration issue

Issue:

As per suggestion of ChatGPT we tried to use `mapbox_gl` for the mapbox integration. But we faced the integration error.

* What went wrong:

A problem occurred evaluating project `':mapbox_gl'`.

> SDK Registry token is null. See README.md for more information.

Impact:

It took us about one day to resolve the issue. We just needed to abandon this package.

Before abandoning it we also tried several LLM (deepseek, claude, grok, qwen) to find the cause. But the error remains as we go on.

Solution:

The journey is pretty interesting for me. At first I needed my resolve to abandon the LLM support completely. Later I went to the official document of the `mapbox_gl` flutter in pub.dev. There I got an important catch. I am just quoting the line here *"If not properly configured the following error will be given :"* So the error is given because the key is not properly set up. Okay fine then ! Let's dive into the key rescue mission ! Before this line there was a link for setting up `mapbox_gl` for Android. I went to the link and found that there are several steps to set up the key. I followed those steps. But in step 3 I got stuck as there is no file in that name in my flutter project. So I could not proceed with the steps. Then I went to github issues as there are some fine answers about the development. One of the issue posts piqued my interest. [HERE](#) is the reference of the issue. From the comment I realized `mapbox_gl` is not for flutter actually. We can use it for native android or iOS development, not for cross platform. Also noticed another thing: the `mapbox_gl` pub dev package was updated 3 years ago. So naturally it should have some issue with the current flutter. Then we just abandoned it and moved to an alternative package named `"mapbox_map_flutter"`. I used the documentation and the process was real quick.

Lesson from this mistake:

We learned an important lesson from this mistake which Sam Altman (CEO of OpenAI) recently told not to blindly accept the suggestions of the chatGPT. If we had followed the official documentation for setting up the mapbox we could have easily fixed the issue in just

one hour to three hours at most. But the issue was delayed to one day as we relied on the GPT. Guess what next? We just abandoned the gpt for major decision making. Though it took a precious day from us, we believe that it saved 40% of our time in the future.

Challenge 6: Windows firewall issue

Issue:

At first we used localhost for the backend and frontend connections. But an important issue arises then. How to connect the wireless debug mode? We then just switched to adb for wireless connection. But the localhost didn't work as expected. The solution was pretty straightforward. We just need to give 0.0.0.0 to get in all the devices. But after that it still did not work.

Impact:

Well it did cost several hours (4-5) hours to fix the issue. No major impact except debugging time cost here.

Solution:

At first we checked the same code on the PC using localhost. The code worked just fine in chrome. So the issue was not with the frontend it must be the backend. Then we used the thunderbolt client (api tester like postman) in VScode. There it was just fine. (Note: for checking whether the server is running we simply put a message "Server is running at full throttle" in the root of the api. The api was working nicely on the computer but it was not working on other devices (Laptop and Mobile phone). Now having lost, we just went to chatGPT to find the error. It gave us the answer to this [post](#). It simply told us to open the 8000 port through the firewall. It did work for us. But for a better one we shifted to macOS (it wants permission before exposing the port).

Challenge 7: Adding style layer in Map

Issue:

The map was showing just fine. But the marker annotation and route was not showing in the map.

Impact:

As I worked mainly on the map, it did hamper the future progress of map integration and styling. Well from my perspective this is a learning flow rather than a challenge.

Solution:

Just went to mapbox flutter sdk documentation for the solution. There indeed I found a process to use. But I personally got disappointed as the solution is not too detailed. So as usual I went to chatGPT to understand the code part. But the first time I didn't get an answer (might be my ignorance or less understanding). I then used the gpt search tool hoping to get a better result. When I did so I found an interesting website. It's name is [deepwiki](#). From what I saw it is an AI powered website for documenting the github repositories. I found some interesting insights about how the package works, its architecture and many more. I am pretty much interested in this website. But it is a shame that I could not deep dive into the website. But I'm gonna to explore this website more in the future.

Challenge 8: Live Location (Still in progress)**Issue:**

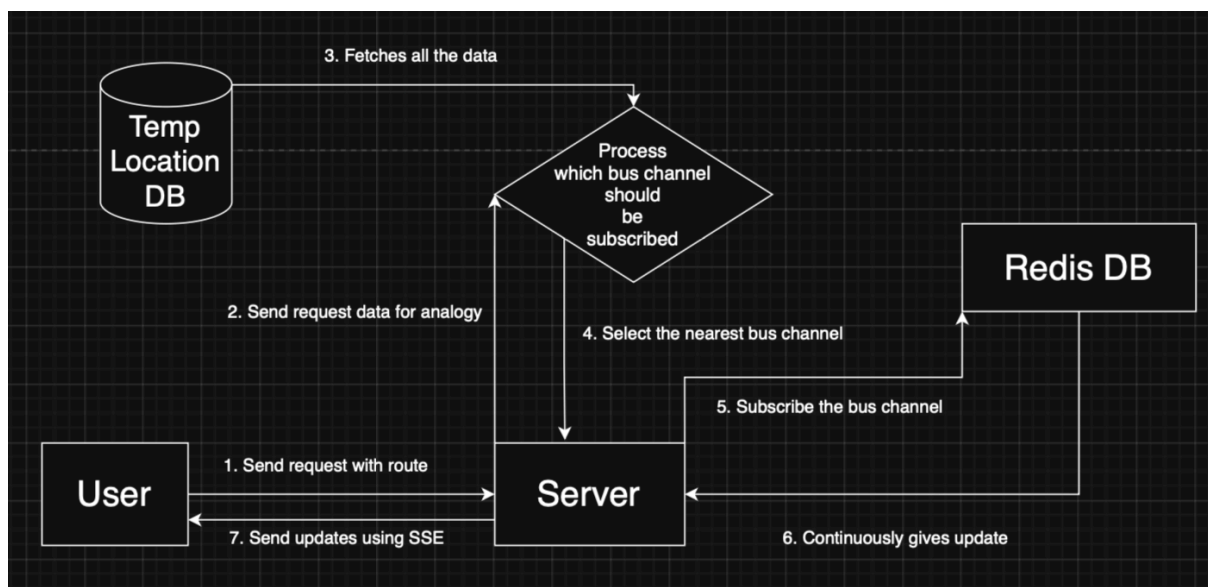
Struggling to how to send the live location using a consistent channel

Impact:

To reduce this impact mainly I am trying parallelly doing the jobs. I got the solution but guess this would take time to implement.

Solution:

Below is a diagram to solve this issue : (Courtesy [draw.io](#))



Though I am working on this architecture I am not confident about finishing this on time. For now just 1, 5, 6 and 7 works just fine. We still need to work on analogy and UI representation. I think we need some guidance to finish this work properly.

Challenge 9: Delayed Code Integration

Issue:

We followed a branching model where each member worked mostly in separate branches, but we did not merge changes back to the main branch frequently. This approach helped avoid immediate conflicts but created uncertainty and larger conflicts when we needed to integrate everything together.

Impact:

Because branches diverged, we faced difficulties when we needed to test features together. Some features depended on other unfinished parts, which delayed full integration and slowed testing on real devices. It also made debugging more difficult because unmerged code was harder to test end-to-end.

Solution:

We realized we needed more frequent check-ins and earlier merging of smaller parts. We are now planning to finish isolated parts first, then integrate them step by step to reduce merge risks. In the future, we will adopt a clearer **feature branch** → **pull request** → **review** → **merge** workflow, to catch issues early and test integrated features sooner.

Challenge 10: Supabase Logic & QR Scanner Data Sync

Issue:

While implementing the ticket booking and validation system, we faced a problem with updating the total ticket after booking in Supabase. When a ticket was “cut” (used), the system was supposed to reduce the total cost in the database. However, the update did not work as expected in Supabase, the value stayed the same.

Interestingly, we tested the same logic with a FastAPI backend, and it worked perfectly, so the issue was likely with Supabase’s client-side or server-side triggers or how we handled updates.

Additionally, when using the QR scanner to validate tickets, we ran into trouble sending the scanned data reliably back to the backend. This caused mismatches between the scanned ticket and the database state.

Impact:

This caused inconsistencies in the ticket booking history and total cost calculations, which could confuse both users and admins. It also slowed down development because the same logic needed to be tested in multiple backend approaches.

Solution:

We decided to handle this by shifting to a more reliable backend for critical operations like FastAPI, where we have full control over logic and state changes.

For the QR scanner, we improved how the scanned data is sent and validated by using cl

Lesson:

Using Supabase for quick development is convenient, but for more complex logic like transactions or atomic updates, having a custom backend (FastAPI) gives us more flexibility and transparency. We learned that some real-time flows need more than just serverless queries — manual checks and clear API design help maintain data consistency and verifying updates immediately.

Individual Responsibilities :

Team Member 1: Sumaiya Rahman Soma (SK-07)

Responsibility: Frontend

- Designed the app logo.
- Built the landing page to fetch user details, get live location, and display the current address.
- Created Sign Up and Log In user interfaces.
- Designed the Booked Ticket page UI with a QR scanner placeholder for future scanning.
- Developed the SOS screen with an emergency call feature.
- Handled runtime permissions for location services.
- Built the Booking History page with data sorting, PDF generation, and a print option for offline use.
- Created the About Us page with app and team member details.
- Applied asynchronous programming to handle location fetching and user data display smoothly.
- Created the database structure for booking tickets and booking history, including natural joins for linking user and ticket data.

Team Member 2: Mir. Md. Ishrak Faisal (FH-12)

Responsibility: Frontend

- Designed the Complaint Box feature, allowing users to select a bus company and submit complaint titles and details through the app.
- Built the Complaint Status Page featuring a button on each complaint card that navigates to a detailed view; the page stores all complaints along with their time, date, and user-submitted title.
- Developed the Current Status page UI where users can check if their complaint is Submitted, Accepted, or Solved.
- Created a social platform within the app where users can view all complaints with anonymous profile pictures, use See More/Show Less options, react with Like buttons, and filter complaints using a search bar.
- Designed My Profile page for all informations including profile picture of a user.
- Built the Edit Profile screen to allow users to update their information and change their profile picture.

- Developed user interface of Settings for managing password.

Team Member 3: Jobaer Hossain Tamim (SH-24)

Responsibility: Map & Backend

- Design:
 - Initially designed Figma Prototype (Later the color, logo and design were revised by the team to a great extent.)
- Frontend :
 - Styling map .
 - Handled user friendly error messages.
 - Handled location permission(used permission_handler).
 - Showing user location.
 - Showing walking path for nearest stoppage.
 - Animated drawer to show all stoppages of a route.
- Backend :
 - Api endpoints for getting all routes, bus stops.
 - Tried to use something like MVC architecture (though could not fully maintain that due to incapability and lack of experience).
 - Fetching the nearest stoppage data from the mapbox api.
 - Currently working on redis pu/sub model to fetch the live location of desired buses only.
- Database :
 - Creating a database for bus stop location.

Team Member 4: Chowdhury Shafahid Rahman (AE-55)

Responsibility: Backend

- Api development with FastAPI:
 - Designed and implemented RESTful endpoints for user complaints, including submission, retrieval, filtering by user, and individual complaint detail fetching.
- Database Management with Supabase:
 - Designed and created the database schema in Supabase, including defining tables such as user_profiles, complaints, bus_company, etc., along with their relationships and constraints.
 - Integrated FastAPI with Supabase (PostgreSQL) using the Supabase Python client.

- Handled CRUD operations on user-related tables (e.g., user_profile, complaints, etc.) while ensuring data consistency.
- Authentication and Authorization:
 - Built token verification mechanisms to authenticate users via JWT tokens obtained from Supabase Auth.
 - Ensured that protected endpoints were only accessible by authorized users.
- Error Handling and Data Validation:
 - Implemented Pydantic models for request and response validation.
 - Handled edge cases with appropriate HTTP status codes and descriptive error messages.
- Testing and Debugging:
 - Used FastAPI's built-in Swagger documentation interface (/docs) to test endpoints, verify request/response formats, and debug API behaviors.
 - Assisted frontend integration by debugging token handling and data mismatch issues.

Future Plan:

In the next two weeks, we plan to combine all individual code branches into a single, fully integrated project. We aim to complete the live map feature so that accurate bus locations are displayed in real time. Additionally, we will work on adding the Find Bus option with detailed routes to help users navigate easily. To support these features, we will update and expand the database structure, ensuring that new routes are properly linked to the Book Ticket system. Some database integration for the Complaint option is also still pending and will be completed during this time. Finally, we plan to add a Forgot Password option to improve account security and make it easier for users to recover their accounts if needed.